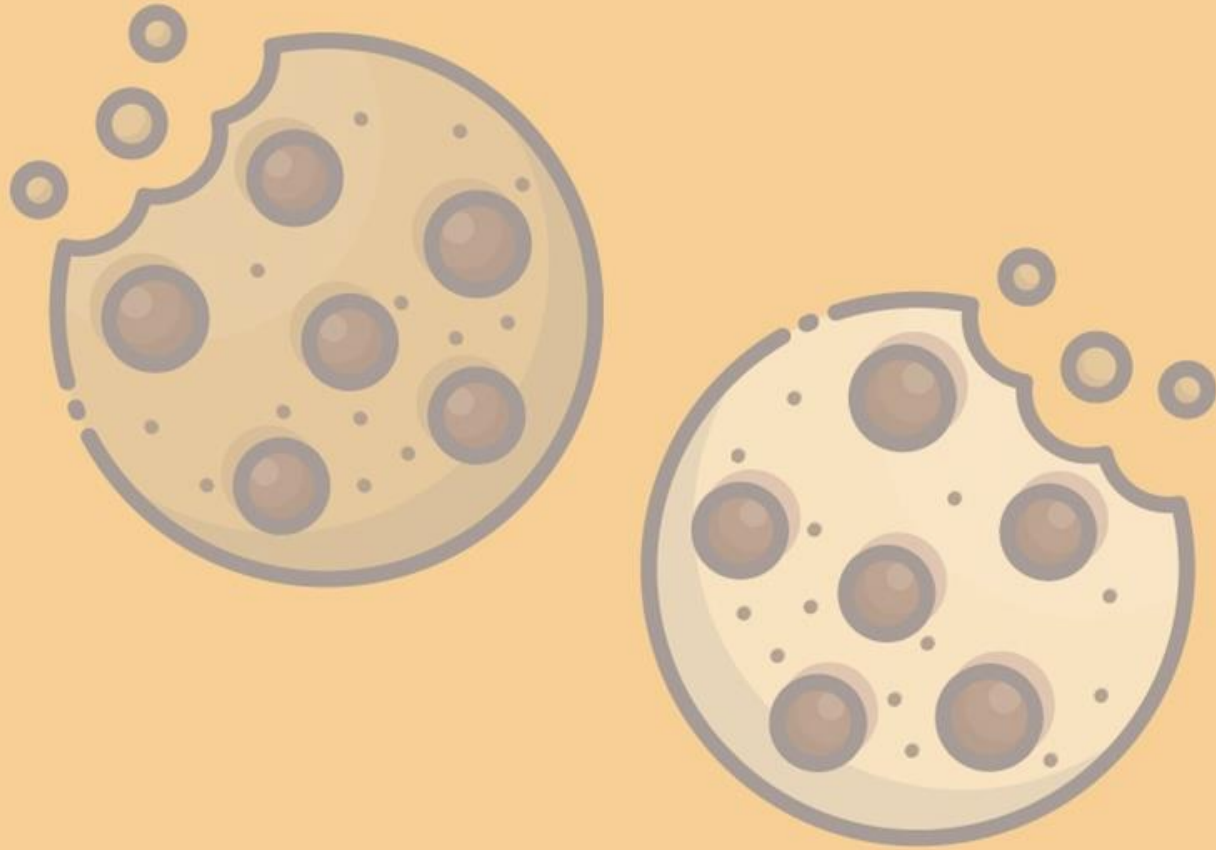




Cookies

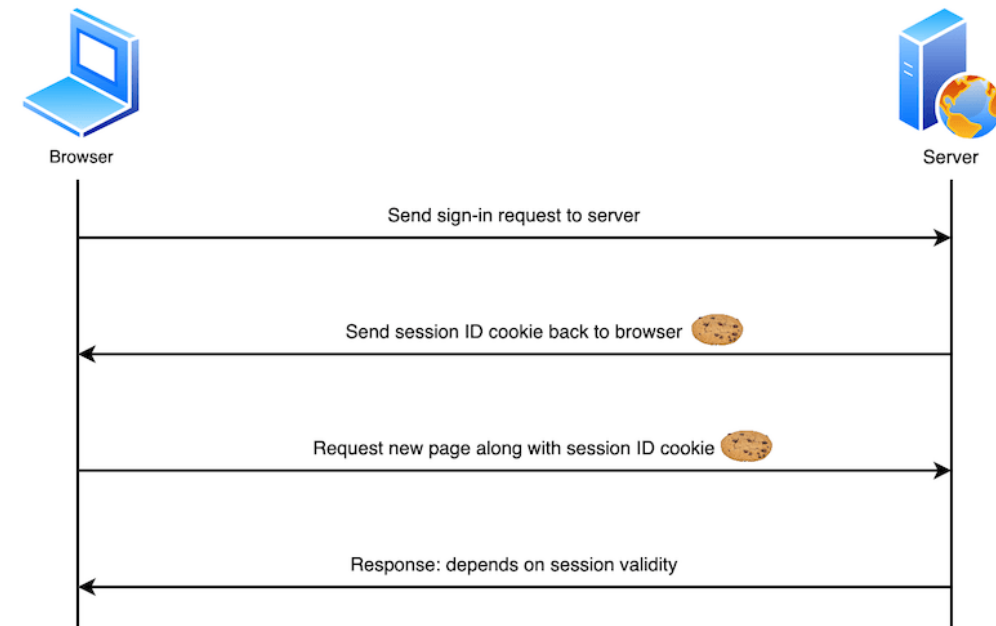
Web storages

- Websites usually store a small amount of data on the browsers.
- There are three main kinds of browser-based storage:
 1. **session storage**- a browser-based storage that saves key-value data for a single tab and automatically clears it when the tab is closed.
 2. **local storage**- a browser-based storage that saves key-value data persistently, allowing it to remain available even after the browser is closed and reopened.
 3. **cookie storage**- small piece of data stored in the browser that can be sent automatically with HTTP requests to the server

Storage Type	Lifetime	Size Limit	Accessible by Server?
Local Storage	Until cleared by user	~5-10 MB	No
Session Storage	Until tab is closed	~5 MB	No
Cookies	Based on expiration date	~4 KB per cookie	Yes (sent with HTTP requests)

Cookies

- A cookie is a piece of data from a website stored within a web browser that the website can retrieve later.
- **Cookies are used to tell the server that users have returned to a particular website.**
- Cookies are sent with every request to the server and are used to store data that needs to persist across multiple requests

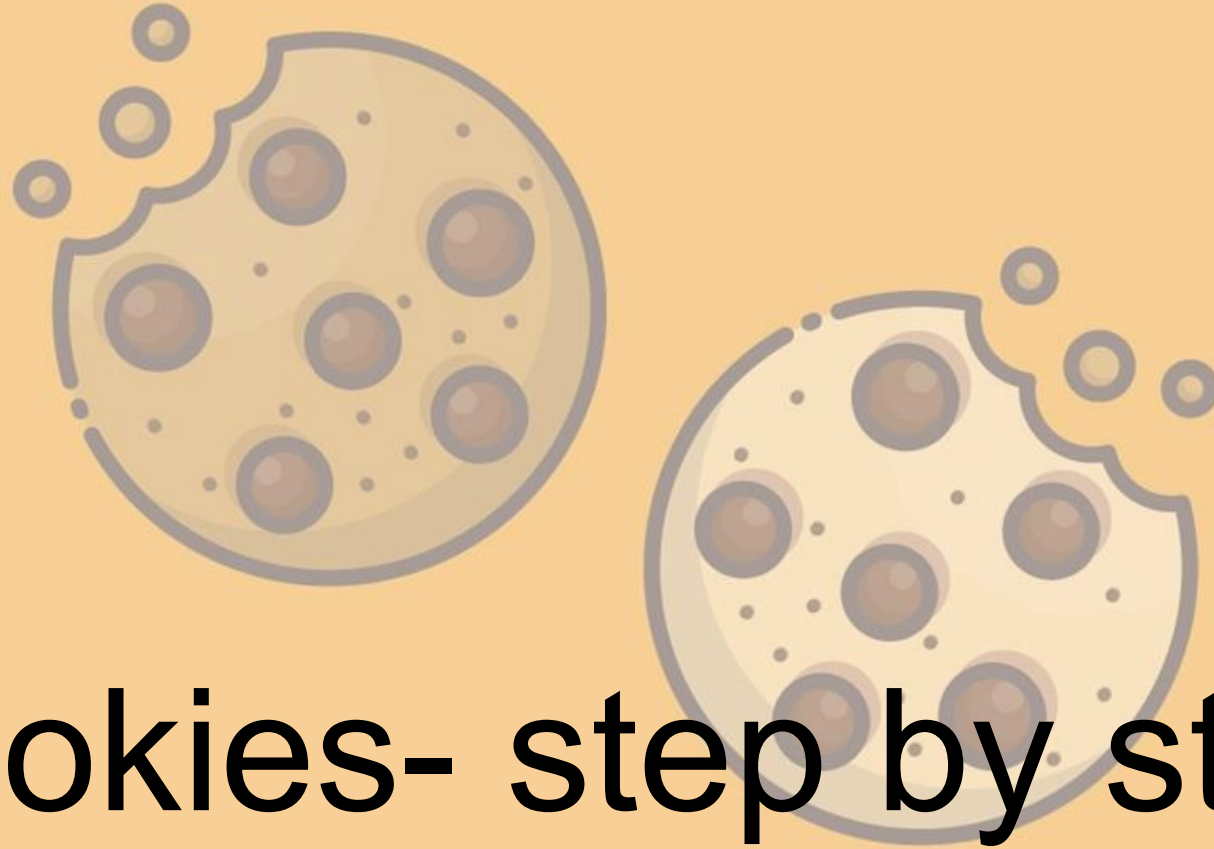


Cookies

- Cookies save users' preferences and authentication.
- For example:
 - **language:** this way, when a user visits that website in the future, the language preference will be remembered.
 - **shopping cart:** a cookie can remember users' choices in the shopping cart and the shopping list item will still be there whenever the user revisits the site.
- Basically, a cookie is used to remember data from the user.

Use cookies safely

- Use cookies securely
- To ensure cookies don't open your app to exploits, **don't use the default session cookie name produced by Express.js** and set cookie security options appropriately.



Cookies- step by step
example

Overview

What is Cookie-Based Authentication?

- Uses cookies to persist user login status across requests.
- A simple way to maintain state in stateless HTTP requests.

Why Use Cookies?

- Persistent across browser sessions.
- Secures sensitive data via attributes like HttpOnly and Secure

Workflow

Step 1- user visits login page

- URL: /login
- Served by GET /login
- HTML form for username and password.

Step 2- user logs in

- URL: /login (via POST)
- Server authenticates user.
- On success, a cookie is set with the username.

Workflow

Step 3- user navigates to profile

- URL: /profile
- Cookie is validated.
- Displays a welcome message if the cookie is valid.

Step 4- user logs out

- URL: /logout
- Clears the cookie and redirects to login.

Install and import packages

```
$ npm init -y
```

```
$ npm i express cookie-parser body-parser
```

Middleware setup

```
const express = require('express');
const cookieParser = require('cookie-parser');
const bodyParser = require('body-parser');
const path = require('path');

const app = express();
const port=3000;

// Middleware
app.use(cookieParser());
app.use(bodyParser.urlencoded({ extended: true }));
```

- cookie-parser: Parses cookies in incoming requests.
- body-parser: Parses form submissions for POST requests.

Login workflow

```
// Route to handle login
app.post('/login', (req, res) => {
  const username=req.body.username;
  const password = req.body.password;

  // Authenticate user
  if (USERS[username] && USERS[username].password === password) {
    // Set a cookie with the username (valid for 7 days)
    res.cookie('authUser', username, {
      httpOnly: true, //Prevents JavaScript access to the cookie.
      secure: false, // Use this with HTTPS in production to prevent cookies from being sent over insecure
connections.
      sameSite: 'Strict', //Protects against CSRF attacks
      maxAge: 7 * 24 * 60 * 60 * 1000, // Ensures the cookie persists across sessions for 7 days.
    });

    res.send(`<h1>Welcome ${USERS[username].name}!</h1><a href="/profile">Go to Profile</a>`);
  } else {
    res.send('<h1>Invalid username or password</h1><a href="/login">Try Again</a>');
  }
});
```

- Extract username and password from req.body.
- Authenticate against the USERS object.
- On success, set a cookie named authUser.

The res.cookie method

```
res.cookie(name, value [, options])
```

- The first parameter is the cookie name.
- The second parameter is the cookie value (This will expire in maxAge time).
- The third parameter is an object containing options about the cookie.

Profile validation

```
// Route to check if the user is authenticated
app.get('/profile', (req, res) => {
  const authUser = req.cookies.authUser;

  if (authUser && USERS[authUser]) {
    // Path to the profile.html file
    const profilePath = path.join(__dirname, 'profile.html');
    // Read the file as UTF-8 text
    fs.readFile(profilePath, 'utf8', (err, data) => {
      if (err) {
        console.error('Error reading profile.html:', err);
      } else {
        // Replace the placeholder with the user's name
        const personalizedProfile = data.replace('<!--USERNAME-->',
]USERS[authUser].name);
        res.send(personalizedProfile);
      }
    });
  } else {
    res.redirect('/login');
  }
});
```

- Extract authUser cookie from req.cookies.
- Check if the cookie value corresponds to a valid user.
- Reads profile page as utf8 charset and replaces the username placeholder
- Redirect to /login if the cookie is missing or invalid.

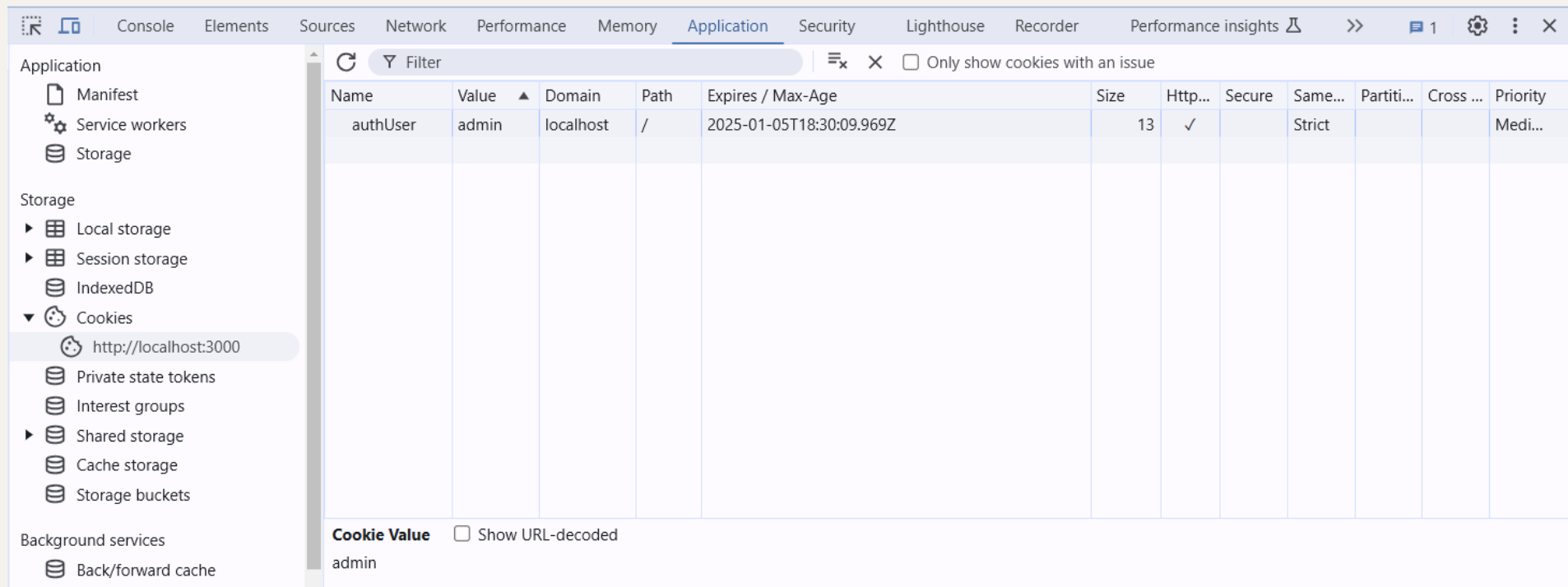
Logout workflow

```
// Route to handle logout
app.get('/logout', (req, res) => {
  res.clearCookie('authUser');
  res.send('<h1>You have been logged out.</h1><a href="/login">Log In</a>');
});
```

- Clear the authUser cookie using res.clearCookie().
- Inform the user they are logged out.

How to inspect the cookie we created?

- Open the developer tool (right click → inspect)
- Navigate to “Application”
- On the left side menu, scroll to “Storage”
- Expand “cookies”
- In the “https://localhost:3000” we will see all cookies used in the site



The screenshot shows the Chrome DevTools Application tab. The left sidebar is expanded to 'Storage' > 'Cookies', with 'http://localhost:3000' selected. The main pane displays a table of cookies. Below the table, the 'Cookie Value' section shows 'admin' with the option 'Show URL-decoded' checked.

Name	Value	Domain	Path	Expires / Max-Age	Size	Http...	Secure	Same...	Partiti...	Cross ...	Priority
authUser	admin	localhost	/	2025-01-05T18:30:09.969Z	13	✓		Strict			Medi...

Cookie Value ☒ Show URL-decoded
admin

Cookie Security Features

- **httpOnly:**
Prevents client-side JavaScript from accessing cookies.
- **secure:**
Ensures cookies are only sent over HTTPS (set true in production).
- **sameSite:**
Prevents cookies from being sent with cross-site requests, mitigating CSRF attacks.
- **maxAge:**
Specifies cookie expiration time in milliseconds (7 days in the example).

httpOnly- Prevent Client-Side Access to Cookies

- Blocks JavaScript from accessing the cookie via **document.cookie**.
- Sensitive cookies (auth tokens) should not be exposed to JS.

How it works?

- Cookie is sent to the server with each request.
- Cookie is **not readable** by JavaScript on the page.
- Always use HttpOnly on authentication cookies and any cookie holding sensitive data.

Secure- Enforce HTTPS-only Cookie Transmission

- Ensures cookie is **only sent over HTTPS** connections.
- Prevents cookie theft over **unencrypted HTTP**.

How it works?

- Cookie is only transmitted when the connection is secure (TLS/SSL).
- Cookie is only transmitted when the connection is secure (TLS/SSL).

sameSite- Mitigate CSRF by Controlling Cross-Site Cookie Sending

- Controls whether cookie is sent in **cross-site requests**.
- Helps prevent **Cross-Site Request Forgery (CSRF)** attacks.
- Provides control over **third-party usage** of your cookies.

What is CSRF?

- A CSRF attack tricks a user's browser into sending an unwanted **request** to a website where the user is already **logged in**.
- The server **thinks the request is legitimate** (because the user's cookies/session are valid), but the user **never intended to perform this action**.

CSRF- Example

- You log into your bank site: mybank.com
- Now your browser holds an auth cookie for mybank.com.
- You visit a malicious site: evil.com
 - That page secretly contains:
 - ``
- Your browser automatically sends this GET request to mybank.com, including your auth cookie (because you're logged in).
- Your bank sees a valid request from you and executes the transfer — without you clicking anything on your bank site!

MaxAge- Define Cookie Lifetime in Seconds

- Defines how long the cookie is valid **after being set** (relative expiration).
- Provides precise control over **cookie expiration time**.

How it works?

- After Max-Age seconds → cookie is automatically deleted by the browser.